FACULDADE ANHANGUERA

Tecnologia em Análise e Desenvolvimento de Sistemas

Convívium Management System

Aplicação integrada para operações de alimentação
Caso Brasa Burger

Gabriel Rodrigues do Prado - 3º semestre
Rafaela Rodrigues Oliveira - 1º semestre

Projeto Integrado I, II e III
Prof. Gilberto Falco Netto

Jundiaí/SP - novembro de 2026

Versão: entrega-2-2026-2-revisao-1

1. Identificação e resumo

Campo	Informação
Instituição	Faculdade Anhanguera
Curso	Tecnologia em Análise e Desenvolvimento de Sistemas
Disciplina e turma	Projeto Integrado I, II e III
Período e local	2026/2, Jundiaí/SP
Docente	Prof. Gilberto Falco Netto
Integrantes	Gabriel Rodrigues do Prado, 3º semestre; Rafaela Rodrigues Oliveira, 1º semestre
Modalidade	Continuidade de protótipo acadêmico

Resumo. O projeto integra consumidor, atendimento e gestão de operações de alimentação. Convívium identifica a plataforma personalizável; Brasa Burger identifica a empresa do caso de aplicação. A interface React/JavaScript usa Supabase Auth e PostgREST, enquanto PostgreSQL executa regras de acesso, integridade e transações. O consumidor seleciona uma unidade, monta o carrinho e envia uma solicitação. A equipe autorizada aceita a solicitação e gera venda e consumo de estoque uma única vez. Programação, algoritmos e matemática aparecem em módulos, validações, estruturas de controle e cálculos monetários e gerenciais. A suíte possui 117 testes aprovados e a instalação integral foi reproduzida em bancos vazios. Conexões PostgreSQL independentes confirmaram idempotência, baixa única e reversão. O resultado é um piloto acadêmico funcional, acompanhado de instruções, evidências e limites explícitos para operação externa.

Palavras-chave: aplicação integrada; JavaScript; algoritmos; matemática computacional; gestão multiunidade.

2. Introdução

Operações de alimentação precisam relacionar pedidos, cadastro, disponibilidade de produtos, estoque e custos. Quando essas informações ficam separadas, a equipe repete registros e encontra dificuldade para recuperar a origem de uma venda ou comparar unidades com os mesmos critérios. O problema deste projeto é integrar esse fluxo com autorização e resultados verificáveis.

Os interessados são consumidores, operadores, gerentes, responsáveis pelo estoque e administradores. No caso Brasa Burger, Japy e Retiro atendem presencial e delivery; Eloy opera somente delivery. O consumidor utiliza o portal da marca e a solicitação chega à unidade escolhida. A plataforma Convívium permite configurar outras empresas, preservando identidade e separação de dados.

A justificativa acadêmica é aplicar programação, decomposição de algoritmos e matemática a uma necessidade operacional. A aplicação oferece exemplos verificáveis de entrada, processamento, persistência e saída. Os resultados relatados são de testes e demonstração; não representam estudo estatístico de produtividade de uma operação comercial.

3. Objetivos e escopo final

O objetivo geral é implementar e documentar uma aplicação integrada para gerir pedidos e informações de operações de alimentação, do consumidor à loja, com regras de acesso, persistência, estoque e cálculo.

Os objetivos específicos são separar plataforma, empresas e unidades; permitir cadastro e solicitação pelo consumidor; receber a solicitação na unidade correta; gerar venda e baixa atômicas no aceite; consultar histórico e indicadores coerentes; e disponibilizar código, testes e instruções correspondentes à apresentação.

Área	Entrega implementada	Limite relevante
Plataforma	Master administra empresas; administrador configura nome, logo, destaque, unidades e equipe	Vínculo de equipe exige conta Auth existente
Acesso	Perfis, empresa e unidade com RLS e validação SQL	Gerente opera somente a unidade vinculada
Consumidor	Portal, conta, catálogo, adicionais, carrinho e acompanhamento próprio	Cadastro público não concede perfil ERP
Operação	Fila por unidade, aceite/recusa, pedido, histórico, cancelamento e reposição	Solicitação aguardando não reserva estoque
Entrega própria	Cotação no servidor, rota e confirmação no mapa	Cobertura depende de endereço e origem válidos
Indicadores	Faturamento, frete separado, ticket, CMV, cobertura e filtros	Custo usa ficha atual; margem não é lucro líquido

Pagamento online, integração automática com marketplaces, financeiro/fiscal completo, aplicativo nativo, convites por e-mail e personalização integral de tema são evoluções. Os canais externos previstos são lançamentos manuais. O frontend e a apresentação interativa estão publicados em HTTPS no GitHub Pages. O consumidor se cadastra com CPF e senha, usando um identificador interno no Auth, sem depender de envio de e-mail. CPF confere consistência matemática, sem comprovar identidade civil. Trajetos físicos e carga da turma ainda exigem ensaio.

4. Histórico e continuidade

A Entrega 1, intitulada Brasa Burguer Management, apresenta problema, proposta, modelagem, organização do grupo e um exemplo JavaScript inicial. Seu ZIP original permanece no repositório. A etapa atual integra portal, contexto de empresas, regras transacionais e documentação de reprodução. A classe inicial e a arquitetura proposta na primeira etapa não descrevem, sozinhas, a implementação atual.

Marco Git	Data	Incremento
`e20ed1b`	13/09/2026	Primeiro registro Git de organização e documentação
`b616dc7`	05/10/2026	Operação, análises e entregas por mapa
`cd50e13` e `1728475`	07 e 08/10/2026	Frete transacional e cancelamento com destinação
`ee20fcf`	17/10/2026	Permissões, regras e testes de banco
`28a283b`	24/10/2026	Portal do consumidor e aceite transacional
`6c83548`	26/10/2026	Empresas, escopos e frete pelo consumidor
`6257e5d`	29/10/2026	Consolidação dos requisitos da Entrega 2
`cb4fb07` e `9efb272`	04/11/2026	Instalação integral, teste nativo e workflow isolado

Os materiais históricos incluem duas apresentações anteriores. As versões atualizadas deste pacote são a apresentação acadêmica e o complemento de fundamentos técnicos. Uma terceira apresentação explica a proposta de modelo de negócio do Convívium, sem inventar faturamento, preços contratados ou resultados de mercado.

5. Método, organização e arquitetura

O trabalho utiliza desenvolvimento incremental, módulos JavaScript, componentes React, SQL/PL/pgSQL, migrações e controle de versões. Node.js 24 executa Vite e Node Test Runner. PGlite verifica regras de banco em ambiente isolado. Um ensaio adicional em PostgreSQL nativo usa conexões independentes para observar concorrência real. O GitHub Actions instala dependências pelo lockfile e executa teste, compilação e auditoria.

O docente aceitou JavaScript, conforme confirmação do responsável em 29/10/2026. A especificação ECMAScript 2025 fundamenta módulos, funções, tipos e estruturas de controle

usados na linguagem (ECMA INTERNATIONAL, 2025). A documentação do PostgreSQL fundamenta transações e controle de concorrência; a documentação Supabase explica a combinação de Auth e políticas de acesso por linha.

Integrante	Organização registrada na Entrega 1	Artefatos relacionados
Gabriel Rodrigues do Prado	Arquitetura, JavaScript, modelagem, integração e revisão técnica	Componentes, módulos, SQL, instalação e documentação técnica
Rafaela Rodrigues Oliveira	Problema, fluxo, entradas/saídas, validação, testes e documentação	Roteiro de uso, casos de teste, relatório e apresentação

Essa tabela recupera a divisão declarada pelo grupo. O histórico usa uma conta Git compartilhada e não comprova autoria individual de cada alteração. Não se atribui um commit específico a um integrante sem registro individual. Ambos precisam explicar o algoritmo, o cálculo e as evidências na apresentação.

Camada	Responsabilidade
Navegador React	Portal, ERP, eventos, estado, validações de apoio e apresentação
Supabase Auth	Contas e sessões. Cadastro público não cria usuário ERP
PostgREST/RPC	Acesso HTTP às consultas e funções autorizadas
PostgreSQL	Dados, RLS, preços, integridade e transações
Edge Function de entrega	Verifica sessão e consulta openrouteservice/HeiGIT com segredo no servidor
OpenStreetMap e Leaflet	Exibem mapa e permitem confirmar o ponto do imóvel

Node.js não atua como API intermediária da aplicação. Os scripts Python são legados independentes de carga, e o diretório MySQL é estudo analítico separado. O guia documenta banco vazio, primeiro master e dados sintéticos. A tabela representa a arquitetura atual; o anexo inclui fontes Mermaid editáveis da arquitetura, do fluxo e do recorte lógico de dados.

6. Integração disciplinar

Disciplina obrigatória	Conceito aplicado	Evidência no projeto
Linguagem de Programação	Tipos, módulos, funções, entrada/saída e erros	`montarSolicitacao`, eventos e envio assíncrono
Algoritmos e Programação Estruturada	Sequência, seleção, repetição e decomposição	Validação de itens/adicionais, fila, aceite e transação

Disciplina obrigatória	Conceito aplicado	Evidência no projeto
Matemática Computacional	Somatórios, proporcionalidade, percentuais e médias	Subtotal, frete, markup, CMV, cobertura e ticket

A matriz do anexo relaciona cada conceito a arquivo, função e teste. A linguagem transforma dados e trata o resultado assíncrono; algoritmos conduzem decisões e repetições; matemática determina valores e o significado dos indicadores. Desenvolvimento em JavaScript, Computação em Nuvem e Análise Orientada a Objetos, presentes na Entrega 1, complementam os três eixos.

As referências herdadas da Entrega 1 incluem Cormen et al. para algoritmos, Rosen para raciocínio discreto e aritmético, e ECMAScript para a linguagem. O enunciado fornecido não contém a relação bibliográfica completa dos planos de ensino. Este relatório identifica as fontes utilizadas e não atribui automaticamente essa seleção à bibliografia institucional das disciplinas.

7. Algoritmos e implementação

O fluxo começa com a escolha da empresa/unidade e a montagem do carrinho. O navegador valida contato e dados básicos. Em delivery, exige cotação válida. O servidor confere sessão, empresa, unidade, modalidade, itens, adicionais, quantidade, preços e cotação antes de persistir.

```
selecionar unidade, modalidade e itens
autenticar consumidor
se delivery: obter cotação válida
validar contato e montar solicitação
enviar com chave idempotente
no servidor:
  para cada item e adicional: conferir dados e preço
  se houver erro: recusar sem persistência parcial
  gravar solicitação aguardando, sem baixar estoque
no aceite da equipe autorizada:
  se já existe venda: devolver a venda existente
  conferir preço, cotação e estoque em transação
  criar pedido, itens e consumo; vincular solicitação
  se houver falha: reverter alterações
acompanhar estado e consultar histórico
```

Sequência organiza etapas. Seleção decide modalidade, acesso e validade. Repetição percorre itens e adicionais. Funções menores separam montagem, cálculo, envio e persistência. O custo de percorrer o carrinho cresce com a quantidade de linhas e adicionais; o ensaio não mede desempenho sob carga.

O trecho real abaixo resume o somatório no navegador. Localização: `frontend/src/lib/portalCliente.js`, função `totalCarrinho`.

```
return itens.reduce((soma, i) => soma +
  (Math.round(Number(i.preco) * 100) +
   i.extras.reduce((s, a) =>
     s + Math.round(Number(a.preco) * 100), 0))
  * i.quantidade, 0) / 100
```

O acumulador externo soma linhas; o interno soma adicionais por unidade. Quantidade multiplica a composição completa. Preços viram centavos antes da soma. O servidor confere o catálogo autorizado, impedindo que preço adulterado no navegador determine o valor persistido.

`criarEnvioTransacional`, em `pedidoTransacional.js`, conserva chave e conteúdo quando a resposta de rede é incerta. No banco, `erp_privado.decidir_mobile` recupera a venda já vinculada. Transação, bloqueios e idempotência impedem baixa duplicada, conforme os testes nativos. RLS restringe leitura por empresa/unidade, e a função verifica autorização antes de alterar dados.

O consumidor acompanha suas próprias solicitações. O cancelamento registra motivo e destino do consumo. A devolução ao estoque ocorre uma vez e preserva o pedido cancelado no histórico.

8. Matemática computacional

O subtotal S soma, em reais, quantidade q de cada linha multiplicada pelo preço p mais seus adicionais a. O total T inclui frete F válido. Quantidades são inteiras positivas. O portal soma centavos; o banco usa valores `numeric` validados.

```
S = soma_i [q_i x (p_i + soma_j a_ij)]
T = arredondar(S + F, 2)
```

Exemplo sintético: item R$ 25,00 e adicional R$ 3,00 geram R$ 28,00. Com frete R$ 3,75, o total é R$ 31,75. A quantidade multiplica também os adicionais. O desconto ilustrativo da Entrega 1 não faz parte desse cálculo atual.

No frete, d é distância de ida por rota em km, m é distância em metros, v é tarifa em R$/km e c é tarifa em centavos/km. Calcula-se `m = arredondar(d x 1000)`, `c = arredondar(v x 100)` e taxa em centavos `arredondar(m x c / 1000)`.

Entrada	Desenvolvimento	Esperado e obtido
3,5 km e R$ 1,50/km	3500 x 150 / 1000 = 525 centavos	R$ 5,25
1,235 km e R$ 1,50/km	185,25 centavos arredondados	R$ 1,85
R$ 24,90 e markup 15%	24,90 x 1,15 = 28,635	R$ 28,64 em JS e SQL

O markup usa inteiros BigInt para evitar a divergência anterior de um centavo no ponto flutuante. Distância e tarifa inválidas são recusadas. O teste matemático comprova fórmula e arredondamento, e não a precisão geográfica de um imóvel.

CMV gerencial usa custo conhecido C dividido pela receita Rf dos itens com ficha conhecida. Ri é a receita de todos os itens, P é o total pago em pedidos não cancelados, Fe é o frete e N é a quantidade desses pedidos.

```
CMV (%) = 100 x C / Rf, se Rf > 0
Margem simplificada (%) = 100 - CMV
Cobertura de custo (%) = 100 x Rf / Ri, se Ri > 0
```

```
Faturamento dos itens = P - Fe
Ticket médio (R$/pedido) = P / N, se N > 0
```

No exemplo validado, dois itens custam R$ 27,90 cada, mas apenas um possui ficha de R$ 8,90. Assim Ri = R$ 55,80, Rf = R$ 27,90 e C = R$ 8,90. CMV = 31,90%, margem simplificada = 68,10% e cobertura de custo = 50%. Frete fica fora da base de CMV. Cancelados ficam fora dos indicadores. Denominador inválido produz resultado indisponível.

O custo usa a ficha atual e pode mudar com a composição. Margem simplificada não inclui todos os gastos e não é lucro líquido. Custo desconhecido não vira zero. A memória anexa detalha unidades, exemplos, limites e localização dos testes.

9. Testes e resultados

A suíte Node.js possui 117 testes aprovados, incluindo 36 testes com PostgreSQL/PGlite. O novo teste de bootstrap instala a sequência integral duas vezes em bancos vazios, sem depender da fixture operacional. Confere primeiro master, ausência de colaborador no cadastro público e proteção dos dados demonstrativos. A compilação Vite passou. A atualização para Vite 6.4.5 corrigiu a vulnerabilidade indicada na auditoria anterior.

O ensaio PostgreSQL nativo de 29/10/2026 instalou a aplicação em dois bancos vazios e usou conexões independentes. A instalação final também passou no job PostgreSQL 17 do GitHub Actions em 04/11/2026. O contrato Auth desse ensaio é local de teste e não simula e-mail.

Caso	Entrada	Esperado	Obtido e interpretação
CT04	Preço 24,90 e markup 15%	28,64 em JS e SQL	Coincidem; regressão aprovada
CT05	Receita 55,80, ficha 8,90 em receita 27,90	CMV 31,90%, margem 68,10%, cobertura 50%	Asserções aprovadas
CT12	Aceite sem estoque ou preço atualizado	Nenhuma venda/baixa parcial	Rollback aprovado
CT19	Bootstrap integral em bancos vazios	Instalar funções e políticas	Duas instalações aprovadas
HM03	Duas conexões aceitando a mesma solicitação	Uma venda 25,00 e baixa 150 g	Mesma venda, saldo 850 g a partir de 1000 g
HM04	Gerente de outra unidade	Leitura vazia e alteração negada	Ambas aprovadas
HM05	Cancelar com devolução	Saldo volta de 850 g a 1000 g	Aprovado, histórico preservado

Em 11/11/2026, a stack Supabase local descartável do GitHub Actions aprovou cadastro por CPF, Auth, PostgREST, PostgreSQL e Realtime reais. As funções do frontend enviaram uma solicitação de dois produtos sintéticos de R$ 27,90, total R$ 55,80. O saldo permaneceu 100 durante a espera, caiu para 98 após dois aceites simultâneos e voltou a 100 no cancelamento. O consumidor recebeu a atualização em preparo pelo Realtime, sem ler o pedido interno. O gerente de outra unidade não leu nem avançou o pedido. Evidência: `anexos/resultado-api-local.json` e execução https://github.com/terpsandlups/BRASA_BURGER_CRUD_ERP/actions/runs/43116523681. Essa evidência é de API e sessões reais locais, sem escrita no Supabase operacional.

Uma revisão da interface pública em 04/11/2026 verificou catálogo, seleção Japy e carrinho com duas unidades de Bacon Cheese: 2 x R$ 26,90 = R$ 53,80. As capturas anexas não mostram envio nem venda. Esse catálogo público difere do produto sintético usado nos testes de banco; os exemplos não são a mesma venda.

O catálogo completo de casos informa entrada, esperado, obtido, interpretação e arquivos. Rejeições esperadas incluem cotação expirada, preço adulterado, CPF inválido, estoque insuficiente e acesso alheio. No Retiro, uma falha anterior de geocodificação bloqueou envio e ofereceu confirmação no mapa, sem taxa inventada.

A publicação HTTPS foi conferida no navegador, incluindo catálogo, unidade, carrinho e cadastro CPF. O ensaio escrito integrado usa funções do frontend e API real isolada. 30 cadastros e solicitações concorrentes passaram no ambiente isolado, com 30 avisos à fila e saldo 100, 70 e 100 após espera, aceite e cancelamento. Trajetos físicos e o ensaio humano continuam pendentes. A escrita sintética no Supabase compartilhado não foi executada. A abordagem isolada preserva o ambiente operacional e explicita o alcance da evidência.

10. Conclusão

O piloto integra consumidor e operação da loja com regras de acesso, cálculo e estoque. As três disciplinas obrigatórias participam do funcionamento e possuem evidências no código e nos testes. O trabalho demonstra decomposição, validação no servidor, transações, idempotência e escolha correta da base dos indicadores.

A separação entre plataforma e empresa permite configurar outras operações sem duplicar a interface. No modelo de negócio proposto, o estabelecimento contrata o software e atende seu consumidor com a própria marca. Considera-se assinatura por empresa/faixa de unidades e serviços opcionais de implantação, treinamento e personalização com escopo. Preços, contratos, custos recorrentes e métricas comerciais ainda precisam de validação e não são resultados obtidos pelo piloto.

O pacote reúne trabalho escrito, apresentação com 10 páginas de conteúdo, anexos, instruções, testes e código identificado por tag. As próximas etapas operacionais são ensaiar o acesso da turma, conferir trajetos reais e dimensionar carga. O portal público e a apresentação interativa acompanham a revisão, com cadastro CPF e atualização por eventos. A pesquisa aprofundada de concorrentes depende do conector Search/Exa. Ensaio oral e submissão institucional são atos posteriores à preparação dos arquivos.

11. Referências

- FACULDADE ANHANGUERA. Projeto Integrado I, II e III, 2026/2: Entrega 2, Aplicação integrada. Enunciado de quatro páginas. Prof. Gilberto Falco Netto. Documento fornecido ao grupo.

- PRADO, Gabriel Rodrigues do; OLIVEIRA, Rafaela Rodrigues. Brasa Burguer Management: proposta e modelagem integrada de um sistema web de gestão multiunidade para food service. Entrega 1. Jundiaí, 2026.

- CONVÍVIUM MANAGEMENT SYSTEM. Código-fonte, testes e documentação. Tag entrega-2-2026-2-revisao-1. https://github.com/terpsandlups/BRASA_BURGER_CRUD_ERP/tree/entrega-2-2026-2-revisao-1. Consulta: 11 nov. 2026.

- ECMA INTERNATIONAL. ECMA-262: ECMAScript 2025 Language Specification. 16. ed., 2025. https://tc39.es/ecma262/2025/. Consulta: 11 nov. 2026. Referência de linguagem e estruturas de controle.

- NODE.JS. Test runner. Documentação da linha 24. https://nodejs.org/docs/latest-v24.x/api/test.html. Consulta: 11 nov. 2026.

- POSTGRESQL GLOBAL DEVELOPMENT GROUP. Transaction Isolation. PostgreSQL 18 documentation. https://www.postgresql.org/docs/18/transaction-iso.html. Consulta: 11 nov. 2026.

- SUPABASE. Row Level Security. https://supabase.com/docs/guides/database/postgres/row-level-security. Consulta: 11 nov. 2026.

- CORMEN, Thomas H. et al. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022. Referência conceitual de algoritmos recuperada da Entrega 1.

- ROSEN, Kenneth H. Discrete Mathematics and Its Applications. 8. ed. New York: McGraw-Hill, 2019. Referência conceitual de matemática recuperada da Entrega 1.

- MELL, Peter; GRANCE, Timothy. The NIST Definition of Cloud Computing. NIST SP 800-145, 2011. Referência de computação em nuvem recuperada da Entrega 1.

- BRASA BURGER. Apresentação demonstrativa de 2026. https://brasa-burguer-apresentacao-2026.azurebard4.chatgpt.site/. Consulta: 11 nov. 2026. Referência visual e narrativa, sem comprovação de métricas operacionais.